LEGAL-TECH AI BRIEFING AGENT

Implementation Specification

Claude Code-Ready · Pydantic Schemas · Agent Workflow · Setup Instructions

Three-Track Intelligence Pipeline: Vendor Updates · Global AI Policy · Thought Leadership

Version 4.0 — March 2026

Claude API (Opus 4.6) · Jina Reader · Spider · Crawl4AI · Microsoft Graph · Tavily

PART I: SYSTEM OVERVIEW

1. Three Briefing Outputs

The system produces three separate email briefings each month, sent from a dedicated Microsoft 365 mailbox (e.g., ai-briefing@betterwiser.com). Each briefing has its own data-gathering strategy, prompt template, and editorial voice.

Briefing	Scope	Primary Sources	Output Format
Track A: Vendor & Customer	Legal-tech vendor updates, Singapore law firm AI adoption, Singapore government / legal association initiatives	Vendor websites, newsletters, Claude web search	Date-led entries with links
Track B: Global AI Policy	AI policy and regulatory updates from APAC, Middle East, EU, UK, US. Prioritised by relevance to Singapore-based legal innovation provider	Government portals, regulator sites, Claude web search	Bold heading + summary + link per update
Track C: Thought Leadership	Key themes in legal AI: change management, workforce transformation, strategic AI adoption, enterprise impact. Curated from named thought leaders and firms	Newsletters, person-specific search, firm insights pages, Tavily deep research	Themed sections with opinion takeaways + BetterWiser relevance

2. The Dummy Inbox Strategy

The agent operates from a dedicated Microsoft 365 mailbox within the firm's tenant. This inbox is subscribed to every relevant industry newsletter, vendor mailing list, and thought leader publication. The inbox is the primary intelligence source — web scraping fills gaps and follows links found in emails.

2.1 Newsletter subscriptions (configure during setup)

Newsletter / Sender	Content Type	Used For Track(s)
Artificial Lawyer	Legal AI news, vendor coverage	A, C
Axiom research updates	In-house legal team AI adoption data	A, C
PwC insights / newsletters	Thought leadership, Budget 2026 analysis	B, C
McKinsey Global Institute	AI adoption, enterprise scaling research	C
Deloitte / EY / KPMG newsletters	Budget analysis, AI workforce commentary	B, C

LevelUp with Ethan Evans (Substack)	Tech leadership, AI strategy	C
Spellbook newsletter	Legal AI tools, Jamie Tso coverage	A, C
Harvey product updates	Vendor product releases	A
Luminance announcements	Vendor partnerships, customer wins	A
vLex / Vincent / Clio updates	Vendor product releases, partnerships	A
Singapore Academy of Law (SAL)	Singapore legal AI policy, events, training	A, B
LegalTech GoWhere	Singapore legal tech ecosystem	A, B
ACC Asia-Pacific	In-house legal conferences, AI governance	A, C

Additional subscriptions should be added as new relevant sources are identified. The agent processes ALL emails in its inbox for the target month — nothing is skipped.

PART II: AGENT ARCHITECTURE

3. Agent Pipeline Overview

The system is a Python application with five sequential phases. Each phase produces typed Pydantic output that feeds into the next. All inter-agent data is strictly typed — no raw dicts or unvalidated JSON. The architecture diagram is provided separately as an inline SVG. See the agent_pipeline_diagram for the visual flow.

3.1 Phase summary

Phase	Name	What It Does	Pydantic Output
1	Trigger	Cron or CLI fires. Loads config YAML. Generates RunContext with month, tracks, recipients.	RunContext
2	Gather	Parallel: (a) read inbox + parse attachments, (b) scrape curated URLs, (c) Claude web search discovery, (d) Tavily deep research for thought leadership, (e) load historical data.	GatheredData
3	Synthesise	Per-track 6-pass pipeline: cluster → triage → draft (extended thinking) → fact-check (Citations API) → grounding verify → format.	SynthesisResult (x3)
4	Validate	Link validation. Grounding threshold check. Optional human review hold.	ValidatedBriefing (x3)
5	Deliver	Send 3 separate emails via MS Graph. Archive to SharePoint. Log confirmations.	DeliveryReceipt (x3)

PART III: PYDANTIC DATA CONTRACTS

4. Core Data Models (schemas.py)

CRITICAL: Every data transfer between pipeline phases MUST use these Pydantic models. Never pass raw dicts, untyped JSON strings, or ad-hoc data structures between agents. This is the #1 cause of pipeline failures in agentic workflows. Validate on write, validate on read.

4.1 Phase 1: Run Context

```
class BriefingTrack(str, Enum):
    VENDOR_CUSTOMER = 'vendor_customer'
    GLOBAL_POLICY = 'global_policy'
    THOUGHT_LEADERSHIP = 'thought_leadership'

class SourceTier(str, Enum):
    PRIMARY = 'primary'              # Company's own blog/announcement
    TIER1_SECONDARY = 'tier1'         # Reputable publication (Forbes, FT)
    TIER2_SECONDARY = 'tier2'         # Aggregator, blog, commentary

class RunContext(BaseModel):
    run_id: str                       # e.g., '2026-02-01T080000'
    target_month: str                 # e.g., '2026-02'
    month_start: date                 # e.g., date(2026, 2, 1)
    month_end: date                   # e.g., date(2026, 2, 28)
    tracks: list[BriefingTrack]       # Which tracks to run
    recipients: dict[BriefingTrack, list[str]]  # Email recipients per track
    sender_email: str                 # The agent's M365 mailbox
    output_dir: Path                  # e.g., runs/2026-02-01T080000/
```

4.2 Phase 2: Gathered Data

```
class ScrapedSource(BaseModel):
    url: str
    title: str
    content_markdown: str             # Clean markdown text
    source_tier: SourceTier
    domain: str                       # e.g., 'harvey.com'
    scraped_at: datetime
    scraper_used: str                 # 'jina' | 'spider' | 'crawl4ai' | 'web_fetch'
    track_tags: list[BriefingTrack]   # Which tracks this source is relevant to
    word_count: int

class EmailSource(BaseModel):
    message_id: str                   # Graph API message ID
    subject: str
    sender: str
    received_date: datetime
    body_markdown: str                # Extracted and cleaned body text
    extracted_links: list[str]         # All URLs found in email body
    attachments: list[AttachmentContent]
    track_tags: list[BriefingTrack]

class AttachmentContent(BaseModel):
    filename: str
    mime_type: str                    # 'application/pdf', 'image/png', etc.
```

```
    extracted_text: str
    extraction_method: str            # 'pymupdf' | 'python-docx' | 'claude_vision' |
'pandas'
    extracted_links: list[str]

class DiscoveredArticle(BaseModel):
    url: str
    title: str
    summary: str                      # 2-3 sentence summary from Claude
    source_tier: SourceTier
    discovered_via: str               # 'claude_web_search' | 'tavily_research' |
'email_link'
    search_query_used: str
    track_tags: list[BriefingTrack]
    published_date: date | None

class GatheredData(BaseModel):
    run_context: RunContext
    scraped_sources: list[ScrapedSource]
    email_sources: list[EmailSource]
    discovered_articles: list[DiscoveredArticle]
    historical_briefings: list[dict]  # Previous months' structured outputs
    stats: GatheringStats

class GatheringStats(BaseModel):
    total_emails_read: int
    total_urls_scraped: int
    total_discoveries: int
    total_attachments_parsed: int
    scrape_failures: list[str]        # URLs that failed all tiers
    duration_seconds: float
```

4.3 Phase 3: Synthesis Models

```
class EventCluster(BaseModel):
    cluster_id: str
    event_summary: str                # 1-sentence description of the event
    primary_source: ScrapedSource | DiscoveredArticle
    corroborating_sources: list[ScrapedSource | DiscoveredArticle]
    track: BriefingTrack
    is_new_entrant: bool              # First time this entity appears
    trend_annotation: str | None      # e.g., 'Third consecutive month'

class BriefingItem(BaseModel):
    item_id: str
    heading: str                      # Bold heading text
    date_str: str                     # e.g., 'On 23 February 2026'
    body: str                         # 1-2 sentence description
    opinion_takeaway: str | None      # For thought leadership items
    betterwiser_relevance: str | None  # For thought leadership items
    source_urls: list[str]            # All URLs cited
    citations: list[dict]             # Citation API objects
    confidence_score: float           # 0.0-1.0 from fact-check pass

class ThemeGroup(BaseModel):          # Thought leadership only
    theme_name: str                   # e.g., 'AI Workforce Transformation'
    theme_description: str
    items: list[BriefingItem]
```

```
class SynthesisResult(BaseModel):
    track: BriefingTrack
    sections: list[BriefingItem] | list[ThemeGroup]  # Items or ThemeGroups
    grounding_report: GroundingReport
    hot_vendor_suggestion: str | None  # Track A only

class GroundingReport(BaseModel):
    total_claims: int
    verified_claims: int
    flagged_claims: list[FlaggedClaim]
    pass_rate: float                  # verified/total

class FlaggedClaim(BaseModel):
    claim_text: str
    claim_type: str                   # 'number' | 'date' | 'entity' | 'unsupported'
    nearest_source_match: str | None
    resolution: str                   # 'corrected' | 'removed' | 'kept_with_caveat'
```

4.4 Phase 4-5: Validation & Delivery

```
class LinkCheckResult(BaseModel):
    url: str
    status_code: int | None
    is_valid: bool
    replacement_url: str | None       # Wayback Machine fallback

class ValidatedBriefing(BaseModel):
    track: BriefingTrack
    html_body: str                    # Final Outlook-compatible HTML
    subject_line: str
    link_check_results: list[LinkCheckResult]
    grounding_pass_rate: float
    ready_to_send: bool               # False if grounding < 95%

class DeliveryReceipt(BaseModel):
    track: BriefingTrack
    message_id: str                   # Graph API message ID
    sent_at: datetime
    recipients: list[str]
    status: str                       # 'sent' | 'held_for_review' | 'failed'
```

PART IV: DATA GATHERING STRATEGY

5. Phase 2: Gathering (Parallel Execution)

All five gathering sub-pipelines run concurrently via asyncio.gather(). Total target: 150–300 pages of content ingested per run.

5.1 Sub-pipeline A: Inbox Processing

Read ALL emails from the agent's inbox for the target month. For each email: extract body text, harvest all hyperlinks, parse all attachments (PDF via PyMuPDF with Claude Vision OCR fallback, DOCX via python-docx, images via Claude Vision, Excel via pandas). Tag each email with relevant track(s) based on sender domain matching.

Every "Read more" link found in newsletter emails is added to the scrape queue for Sub-pipeline B.

5.2 Sub-pipeline B: Tiered Web Scraping

Scrape all curated URLs plus all links harvested from emails. Tiered fallback: Jina Reader (free) → Spider ($0.48/1K pages) → Crawl4AI (self-hosted fallback). Each source is tagged with its authority tier from the config lookup table.

Curated URL lists (configure in YAML)

Vendor / Entity	URL to Scrape	Track + Tier
Harvey	harvey.com/blog, harvey.com/product-updates	A — Primary
Legora	legora.com/blog, legora.com/news	A — Primary
vLex / Vincent / Clio	vlex.com/blog, clio.com/blog	A — Primary
Luminance	luminance.com/news	A — Primary
SpotDraft	spotdraft.com/blog	A — Primary
Spellbook	spellbook.legal/blog	A — Primary
Jigsaw	jigsaw.xyz/blog (verify URL)	A — Primary
Anthropic (Claude legal)	anthropic.com/news, claude.ai/blog	A — Primary
EU AI Office	digital-strategy.ec.europa.eu/en/policies/ai-office	B — Primary
UK ICO	ico.org.uk/about-the-ico/media-centre	B — Primary
SAL (Singapore)	sal.org.sg, lawgazette.com.sg	A, B — Primary
PwC Singapore	pwc.com/sg/en/insights.html	C — Tier-1 Secondary
McKinsey	mckinsey.com/capabilities/mckinsey-digital/our-insights	C — Tier-1 Secondary

5.3 Sub-pipeline C: Claude Autonomous Discovery

Claude's web_search tool runs targeted queries per track. This catches articles not in your curated lists or newsletters. Estimated: 40–60 searches across all tracks.

Track A discovery queries (examples)

- "Harvey legal AI product update February 2026"
- "Legora legal AI news February 2026"
- "Singapore law firm AI adoption 2026"
- "new legal AI startup funding 2026" (hot vendor discovery)
- "legal AI vendor partnership announcement February 2026"

Track B discovery queries (examples)

- "APAC AI regulation update February 2026"
- "EU AI Act enforcement update February 2026"
- "UK AI governance policy 2026"
- "Middle East AI legal policy Dubai Abu Dhabi 2026"
- "US AI risk management framework financial services 2026"
- "Singapore AI policy Budget 2026"

5.4 Sub-pipeline D: Thought Leadership Deep Research (6 waves)

This is the deepest research phase. It runs six sequential waves designed to maximise the number of unique thought leadership sources discovered. Since cost is not a constraint, this phase uses 80–150 web searches and scrapes 50–100 additional pages beyond the curated lists.

Wave 1 — Newsletter extraction

Extract all thought leadership content from inbox newsletters. Identify every named person, every named firm, and every linked article. Build a dynamic watchlist of people and firms mentioned this month.

Wave 2 — Person-specific deep search (4+ searches per person)

For each person on the watchlist, Claude runs multiple targeted searches:

- "Jamie Tso" legal AI February 2026
- "Jamie Tso" site:medium.com OR site:substack.com
- "Jamie Tso" legalquants hackathon
- "Jamie Tso" conference OR keynote OR panel 2026

Repeat for: Anna Lozynski / Anna Guo, Patrick Tan, Damien Riehl, Bob Ambrogi, and any new names discovered in Wave 1. Estimated: 40–60 searches.

Wave 3 — Firm thought leadership pages (direct scrape)

Scrape the dedicated insights/thought-leadership sections of: PwC (global + Singapore), McKinsey Global Institute, EY, Deloitte, KPMG, BCG, Artificial Lawyer. Use Spider with depth=2

to crawl each firm's insights section. Claude filters for legal/AI relevance. Estimated: 30–50 pages scraped.

Wave 4 — Tavily deep research for theme discovery

Use Tavily's /research endpoint with broad thematic queries. Tavily runs a multi-step search workflow, analyses results, and returns structured reports with citations. This catches thought leadership from people and firms NOT on your watchlist.

- "AI workforce transformation legal profession 2026"
- "enterprise AI adoption maturity legal services"
- "change management generative AI law firms"
- "strategic AI value creation professional services 2026"

Estimated: 10–20 Tavily research calls.

Wave 5 — Semantic similarity expansion

For the top 5–10 best articles found in Waves 1–4, use Claude's web search to find conceptually related content. Prompt: "Find articles published in February 2026 that discuss similar themes to this article: [title + summary]." This catches articles using different terminology that keyword searches would miss.

Wave 6 — Conference and event speaker mining

Scrape agendas and speaker lists from: Legal Innovation Festival Southeast Asia 2026, ACC Asia-Pacific Annual Conference, ILTACON, other events in the target month. Search for each speaker's recent publications. This surfaces thought leaders your watchlist doesn't yet include.

5.5 Sub-pipeline E: Historical Data Loading

Load structured JSON outputs from the previous 2–3 months' runs for cross-month trend detection (new entrants, momentum, dropped topics, valuation trajectories).

PART V: SYNTHESIS PIPELINE (PER TRACK)

6. Six-Pass Synthesis (runs once per track)

The synthesis pipeline runs independently for each of the three tracks, using the same 6-pass architecture but with track-specific prompt templates and output formats.

Pass	Name	Purpose	Output Type
0	Cluster + dedup	Group sources by event. Identify duplicates. Load trends. Auto-detect hot vendors (Track A). Identify dominant themes (Track C).	list[EventCluster]
1	Triage	Sort clusters into track sections. Rank by authority tier. Ensure every source is accounted for. Flag gaps.	list[EventCluster] (sorted)
2	Draft	Generate full briefing text. Extended thinking enabled. Apply track-specific prompt template. Write date-led entries (A, B) or themed sections (C). Include opinion takeaways and BetterWiser relevance (C).	list[BriefingItem] | list[ThemeGroup]
3	Fact-check	Separate Claude call with Citations API. Verify every claim against raw sources. Score confidence. Flag unsupported claims.	list[BriefingItem] (annotated)
3.5	Ground verify	Programmatic: regex + fuzzy match. Extract all numbers, dates, proper nouns. Verify each against source corpus. Flag unmatched.	GroundingReport
4	Format	Generate Outlook-compatible HTML. Validate all links (HEAD requests). Replace dead links with Wayback fallbacks. Apply email styling.	ValidatedBriefing

6.1 Prompt Templates (from coworker's actual workflow)

These templates are derived from the exact prompts used by the team member who previously did this work manually. They encode her editorial judgment, formatting preferences, and prioritisation criteria. The date range is parameterised; everything else is preserved.

Track A: Vendor & Customer Updates prompt

System: You are an expert legal technology industry analyst. Your task is to generate a comprehensive vendor and customer update briefing.

User: Focus on 3 customer segments all in the month of {target_month_human}: (i) legaltech vendors (focus primarily on Harvey, Legora, vLex/Vincent, Luminance, SpotDraft, Spellbook, Jigsaw, Claude legal AI, and any hot new vendors you discover) then (ii) law firm and in-house legal team updates from within Singapore, and (iii) government and legal association updates from within Singapore.

Format: Start each descriptive sentence with the date, e.g., “On 10th February, xyz...” and end the statement with accurate links relied upon. Use a bullet for each update. List the top 10–15 most significant updates. If a notable new vendor has emerged that is not on the core watchlist, include a brief recommendation at the end.

Track B: Global AI Policy prompt

System: Act as an expert legal industry watcher, researcher and analyst.

User: Generate a list of all updates published from {month_start} to {month_end} related to global AI legal policy updates. Focus on APAC, Middle East, EU, UK, US. Exclude updates published prior to the reporting period. Use websites of credible and relevant policy players.

Format: List up to 6–8 relevant updates. Prioritise in descending order based on relevance and materiality (importance to a provider of legal innovation services based in Singapore). Each update starts with a bold heading, followed by a 1–2 sentence summary, and an embedded hyperlink. Example: Ministry of Law publishes draft Guide on GenAI. On 1 September, the Ministry of Law published a draft Guide for Using Generative AI in the Legal Sector. It invites comments on the draft Guide from 1 September to 30 September. (link)

Track C: Thought Leadership prompt

System: You are a senior legal technology strategist curating thought leadership for BetterWiser, a Singapore-based legal innovation services provider.

User: Identify key themes in legal technology focusing on: (1) change management and workforce transformation in the age of GenAI, (2) strategic AI adoption, value creation, and enterprise impact, (3) AI usage maturity, and (4) any other key themes on which leading thought leaders have commented. Focus on pieces from PwC, McKinsey, EY, Deloitte, Artificial Lawyer, Anna Lozynski, Patrick Tan, Jamie Tso (Legal Quants), and other people with strong insights.

Format: Group items by theme. Each item starts with a bold heading, followed by the date and description ending with a link. Include an ‘Opinion takeaway’ paragraph after each item explaining its significance. Include a ‘Relevance to BetterWiser’ note for each theme section.

BetterWiser context: [INSERT BETTERWISER COMPANY DESCRIPTION HERE — services, target clients, competitive positioning, strategic priorities. This is the only manual input needed before deployment.]

6.2 Auto-Expanding Vendor Watchlist (Track A)

Each run includes a dedicated discovery search for emerging vendors: “new legal AI startup 2026” / “legal AI funding round {target_month}” / “emerging legal technology vendor 2026.” Any vendor appearing in 3+ independent sources but not on the current watchlist is flagged with a one-paragraph recommendation at the end of the Track A briefing.

PART VI: SETUP & DEPLOYMENT

7. Prerequisites

Requirement	Action	Notes
Python 3.12+	Install via pyenv or system package manager	Required for asyncio improvements and type hints
Anthropic API key	Create at console.anthropic.com	Needs Opus 4.6 access. Set ANTHROPIC_API_KEY env var
Azure AD app registration	Register in portal.azure.com > App registrations	Grant Mail.Read + Mail.Send (Application). Get admin consent
Dedicated M365 mailbox	IT creates ai-briefing@yourfirm.com	Must have Exchange Online licence. Subscribe to all newsletters
Spider API key	Create at spider.cloud	Pay-as-you-go, no subscription. Set SPIDER_API_KEY env var
Tavily API key	Create at tavily.com	Free tier: 1000 credits/month. Set TAVILY_API_KEY env var
BetterWiser company brief	Write 500-1000 word description of services, clients, strategy	Place in config/betterwiser_context.txt
Docker (for Crawl4AI)	Install Docker Desktop or Docker Engine	Only needed for Tier 3 fallback scraper

8. Environment Variables

```
# .env file (never commit to git)
ANTHROPIC_API_KEY=sk-ant-...
AZURE_TENANT_ID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
AZURE_CLIENT_ID=xxxxxxxx-xxxx-xxxx-xxxx-xxxxxxxxxxxx
AZURE_CLIENT_SECRET=your-client-secret
AGENT_MAILBOX=ai-briefing@yourfirm.com
SPIDER_API_KEY=your-spider-key
TAVILY_API_KEY=tvly-your-key
```

9. Project Structure

```
legal-briefing-agent/
├── config/
│   ├── briefing_config.yaml         # Source lists, queries, recipients, tiers
│   ├── betterwiser_context.txt       # Company description for Track C
│   ├── newsletter_subscriptions.yaml # Sender-to-track mapping
│   ├── vendor_watchlist.yaml         # Current vendor list + authority tiers
│   └── prompt_templates/
│       ├── track_a_vendor_customer.txt
│       ├── track_b_global_policy.txt
│       └── track_c_thought_leadership.txt
├── src/
│   ├── orchestrator.py               # Main entry: async phase sequencing
```

```
│   ├── schemas.py                    # ALL Pydantic models (Part III above)
│   ├── gatherers/
│   │   ├── inbox_reader.py           # MS Graph: read emails + parse attachments
│   │   ├── attachment_parser.py       # PDF, DOCX, image, Excel extraction
│   │   ├── web_scraper.py            # Tiered: Jina → Spider → Crawl4AI
│   │   ├── discovery.py              # Claude web_search per track
│   │   ├── thought_leadership.py     # 6-wave deep research (Waves 1-6)
│   │   └── history_loader.py         # Load previous months' outputs
│   ├── synthesis/
│   │   ├── pass0_cluster.py          # Dedup + trends + theme detection
│   │   ├── pass1_triage.py           # Sort into sections + authority rank
│   │   ├── pass2_draft.py            # Deep synthesis w/ extended thinking
│   │   ├── pass3_factcheck.py        # Citations API fact verification
│   │   ├── pass35_grounding.py       # Programmatic fuzzy-match verification
│   │   └── pass4_format.py           # HTML email generation + link check
│   ├── delivery/
│   │   ├── email_sender.py           # MS Graph: send 3 briefing emails
│   │   └── archiver.py               # SharePoint upload
│   └── utils/
│       ├── authority.py              # Source tier lookup from config
│       ├── grounding.py              # Fuzzy-match engine
│       ├── email_parser.py           # HTML-to-markdown + link extraction
│       ├── retry.py                  # Exponential backoff retry decorator
│       └── logging_config.py         # Structured JSON logging
├── runs/                                   # Timestamped output per run
│   └── 2026-02-01T080000/
│       ├── raw_data/                 # Phase 2: all gathered content
│       ├── synthesis/                # Phase 3: all 6 pass outputs x3 tracks
│       └── delivery/                 # Phase 5: sent email confirmations
├── tests/
├── requirements.txt
├── .env                                    # Never commit
└── Dockerfile                              # For Crawl4AI fallback
```

10. requirements.txt

```
anthropic>=1.0.0
azure-identity>=1.15.0
msgraph-sdk>=1.2.0
pydantic>=2.6.0
spider-client>=0.1.0
tavily-python>=0.5.0
crawl4ai>=0.8.0
pymupdf>=1.24.0
python-docx>=1.1.0
openpyxl>=3.1.0
pandas>=2.2.0
beautifulsoup4>=4.12.0
html2text>=2024.0.0
fuzzywuzzy>=0.18.0
python-Levenshtein>=0.25.0
httpx>=0.27.0
pyyaml>=6.0.0
python-dotenv>=1.0.0
weasyprint>=62.0
```

11. Running the Agent

```
# First run: install dependencies
```

```
pip install -r requirements.txt

# Monthly scheduled run
python -m src.orchestrator --month 2026-02

# Run a single track only
python -m src.orchestrator --month 2026-02 --track thought_leadership

# Dry run (gather + synthesise but don't send emails)
python -m src.orchestrator --month 2026-02 --dry-run

# Resume from a previous failed run
python -m src.orchestrator --resume runs/2026-02-01T080000/
```

PART VII: COST, MONITORING & SECURITY

12. Cost Per Run (3 tracks)

Component	Usage	Standard	Batch API
Claude tokens (6 passes x3 tracks)	~600K in, ~80K out	$15–$20	$8–$10
Claude web searches (discovery)	150–250 searches	$1.50–$2.50	$1.50–$2.50
Tavily deep research	10–20 research calls	$0.50–$1.00	$0.50–$1.00
Jina Reader (Tier 1 scraping)	~100 pages	$0.00	$0.00
Spider (Tier 2 scraping)	~50 pages	$0.02	$0.02
Microsoft Graph	Read + send	$0.00	$0.00
Total per run		**$17–$24**	**$10–$14**

13. Monitoring & Alerting

- Run duration tracking (alert if >20 minutes for all 3 tracks).
- Source coverage per track (alert if <80% of curated URLs return data).
- Grounding pass rate per track (target: >95%). If below, hold for human review.
- Citation coverage rate per track (target: >90% of factual claims cited).
- Email delivery confirmation (alert if any of 3 sends fails).
- Cost tracking per run (log all token usage, search counts, scraper usage).
- Hot vendor detection flag (log when new vendors are recommended for Track A).

14. Security

- All API keys in environment variables or Azure Key Vault. Never in code or config YAML.
- Azure AD app has least-privilege: Mail.Read + Mail.Send only.
- Raw data auto-purged after 90 days. Structured outputs retained for trend analysis.
- OAuth 2.0 client credentials flow. No browser cookies or session tokens stored.
- All API calls over HTTPS. No inbound ports exposed.
- The agent's mailbox should have restricted login access (service account, no interactive login).

Appendix A: API Quick Reference

API	Endpoint / Feature	Docs
Claude Messages	POST /v1/messages	platform.claude.com/docs
Claude Citations	citations: {enabled: true} on document blocks	platform.claude.com/docs/.../citations
Claude Web Search	Tool: web_search_20260209	platform.claude.com/docs/.../web-search-tool
Claude Web Fetch	Tool: web_fetch	platform.claude.com/docs/.../web-fetch-tool
Claude Batch	POST /v1/messages/batches	platform.claude.com/docs/.../batch-processing
Jina Reader	GET https://r.jina.ai/{url}	jina.ai/reader
Spider	POST /crawl, /scrape	spider.cloud/docs
Crawl4AI	Python: AsyncWebCrawler.arun(url)	github.com/unclecode/crawl4ai
Tavily Search	POST /search (basic/advanced)	docs.tavily.com
Tavily Research	POST /research (multi-step)	docs.tavily.com
MS Graph Mail	GET /users/{id}/messages, POST sendMail	learn.microsoft.com/.../user-sendmail
MS Graph Attachments	GET /messages/{id}/attachments	learn.microsoft.com/.../message-list-attachments

Appendix B: What You Need to Provide Before Deployment

The system is fully autonomous once deployed. Only two manual inputs are needed before the first run:

- **1. BetterWiser company description (500–1000 words):** Place in config/betterwiser_context.txt. Describe: what services BetterWiser provides, who your target clients are, your competitive positioning in the Singapore legal innovation market, and your strategic priorities for 2026. This is injected into the Track C prompt so Claude writes genuinely relevant “Relevance to BetterWiser” commentary.
- **2. Newsletter subscriptions:** Subscribe the agent’s mailbox (ai-briefing@yourfirm.com) to all newsletters listed in Section 2.1. This is a one-time setup that takes ~15 minutes.

End of implementation specification.